Kodigo
ACADEMIA DE TECNOLOGÍA CREATIVA

Bootcamp: **Full Stack Junior**

Modulo 4: Profesionalización Fronted con TypeScript y Next.js

Semana 15

Contenido

1. *Server Actions para mutaciones de datos y formularios.*

2. *Gestión de errores con error.js y validación en servidor.*

3. *Web Vitals y optimización avanzada (Streaming, Suspense).*

Contenido:

1. Server Actions para mutaciones de datos y formularios.

1.1. ¿Qué son las Server Actions?

Las Server Actions en Next.js (App Router) son funciones asincrónicas que se ejecutan exclusivamente en el servidor, pero pueden ser invocadas directamente desde componentes React, especialmente a través de formularios.

Permiten ejecutar lógica backend —como inserciones en base de datos, validaciones o autenticación— sin necesidad de crear endpoints API explícitos como se hacía tradicionalmente.

Se declaran utilizando la directiva:



Ilustración 1 – directiva 'use server' - Fuente: Autor de contenidos

Esto indica que la función debe ejecutarse únicamente del lado del servidor, incluso si es llamada desde un componente cliente.

1.2. Diferencia entre API Routes tradicionales y Server Actions

Antes de Server Actions, en versiones previas de Next.js, el flujo típico era:

1. Crear un endpoint en `/api`
2. Hacer `fetch()` desde el frontend
3. Manejar estados manualmente
4. Revalidar datos

Con Server Actions:

- No se necesita endpoint explícito.
- No se requiere `fetch`.
- Se puede usar directamente `<form action={serverFunction}>`.
- La ejecución es segura por defecto en el servidor.
- Integración natural con TypeScript.

Esto reduce significativamente el boilerplate y simplifica la arquitectura.

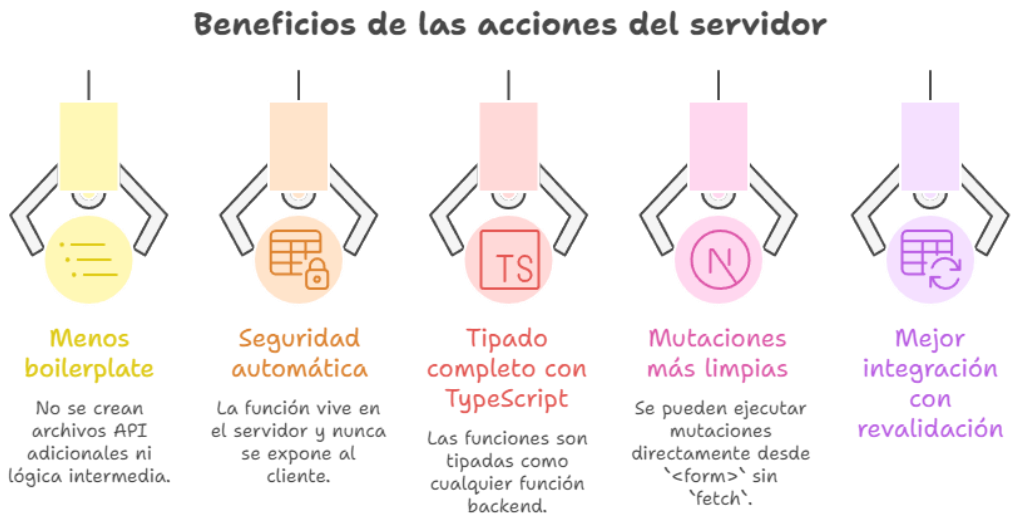


Ilustración 2 – Ventajas de Server Actions - Fuente: Autor de contenidos

1.3. Casos de uso comunes Server Actions

Uso	Ejemplos
Envío de formularios	Registro de usuarios, comentarios, contacto.
Creación / edición de registros	CRUD en base de datos.
Autenticación	Login, logout, manejo de sesiones.
Mutaciones con base de datos	Insert, update, delete con ORM o cliente SQL.

Tabla 1 - Uso de server actions - Fuente: Autor de Contenidos

1.4. Ejemplo de Implementación Server Actions

```
'use server'

import { revalidatePath } from 'next/cache'

export async function createUser(formData: FormData) {
  const name = formData.get('name') as string

  // Simulación de guardado en base de datos
  await db.user.create({
    data: { name }
  })

  // Revalidar para evitar stale UI
  revalidatePath('/users')
}
```

Ilustración 3 – Ejemplo de server actions - Fuente: Autor de contenidos



```
import { createUser } from '@/app/actions'

export default function UserForm() {
  return (
    <form action={createUser}>
      <input name="name" placeholder="Nombre" required />
      <button type="submit">Crear Usuario</button>
    </form>
  )
}
```

Ilustración 4 – Ejemplo implementación server actions - Fuente: Autor de contenidos

Concepto Clave: Consistencia de Datos

Cuando ocurre una mutación:

1. Se actualiza la base de datos.
2. Se invalida el caché.
3. Se vuelve a renderizar el componente del servidor.
4. La UI refleja datos actualizados.

Esto evita interfaces obsoletas (*stale UI*) y mantiene sincronización entre cliente y servidor sin usar estados globales complejos.

2. Gestión de errores con error.js y validación en servidor.

En el App Router de Next.js, el manejo de errores se basa en una arquitectura jerárquica por segmentos de carpeta. Cada carpeta puede contener archivos especiales que actúan como capas de protección alrededor de sus componentes.

Piensa en esto como un edificio con sistemas de seguridad por piso: si algo falla en el piso 3, no necesariamente se incendia todo el edificio. Solo ese segmento queda aislado.

Archivos clave:

- error.js - Captura errores del mismo segmento.
- global-error.js - Captura errores críticos a nivel raíz.
- not-found.js - Maneja errores 404 controlados.
- loading.js - Maneja estados de carga.

Jerarquía de burbujeo

Cuando ocurre un error:

1. Next.js busca un `error.js` en el mismo segmento.
2. Si no existe, sube al segmento padre.
3. Si llega al root, usa `global-error.js`.

Este sistema evita que toda la aplicación colapse por un error aislado.

Archivo	Nivel de Captura	Caso de Uso Principal
error.js	Segmento local	Errores en Server/Client Components del mismo nivel
global-error.js	Root Layout	Errores críticos globales
not-found.js	Segmento / Global	Manejo de rutas inexistentes
loading.js	Segmento local	Estados de carga

Tabla 2 - archivos errors y loading en Nextjs 16+ - Fuente: Autor de Contenidos

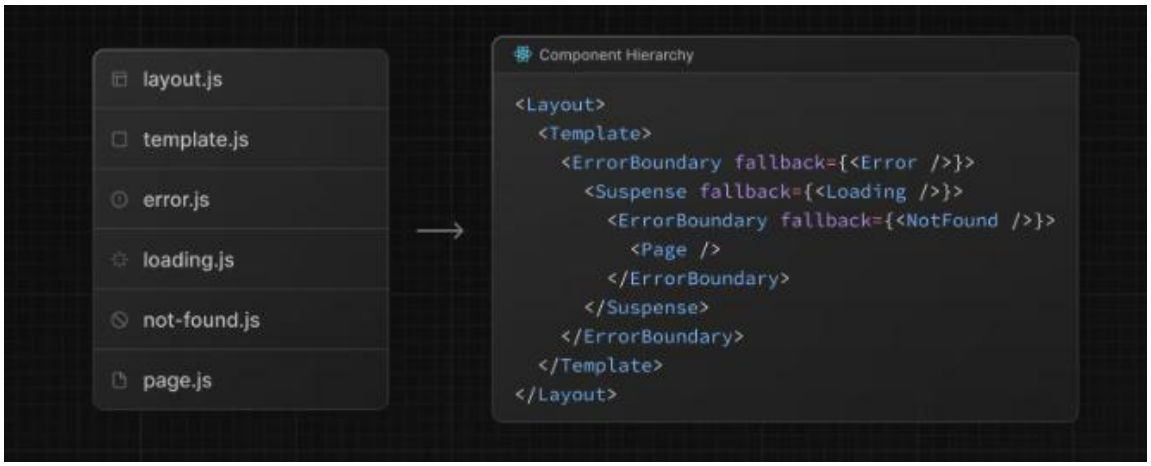


Ilustración 5 – árbol de carpetas de Next.js - Fuente: Nextjs

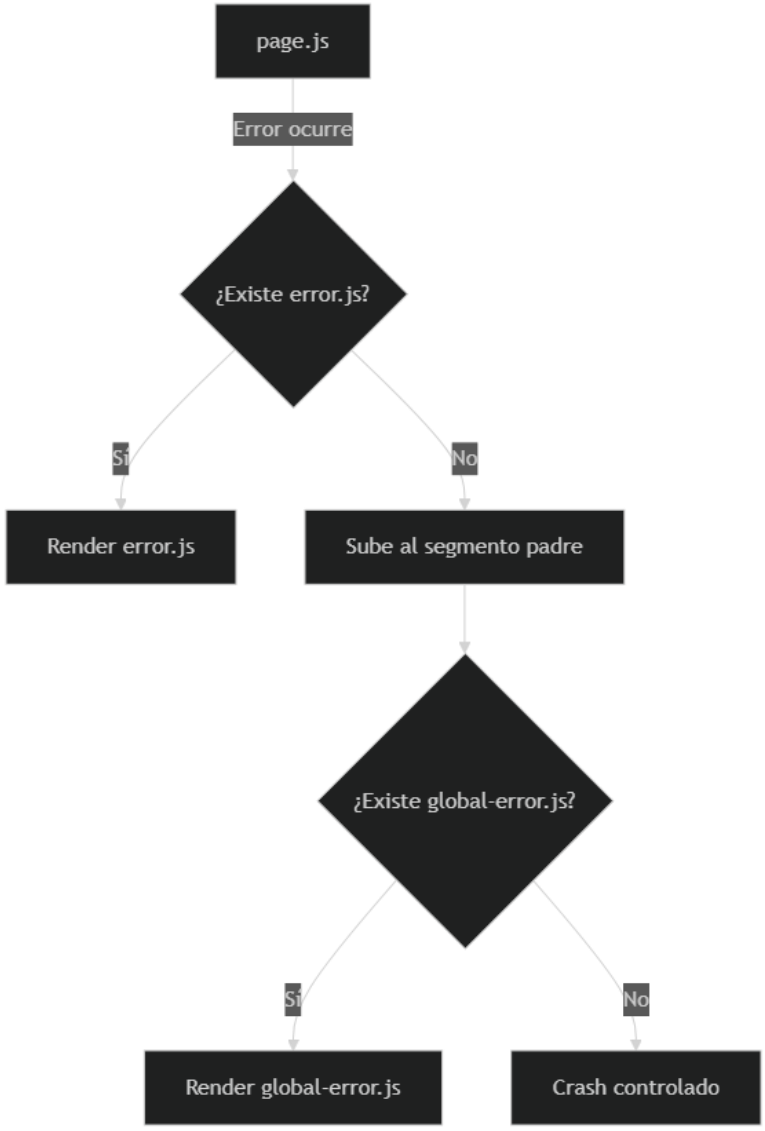


Ilustración 6 – Diagrama de flujo de estructura de funcionamiento de NextJS - Fuente: Autor de contenidos

2.1. Estrategias de Manejo: Errores Esperados vs Inesperados

Errores Esperados

Son errores de validación o lógica:

- Email inválido
- Contraseña incorrecta
- Registro duplicado

Se deben manejar como parte del flujo normal.

Errores Inesperados

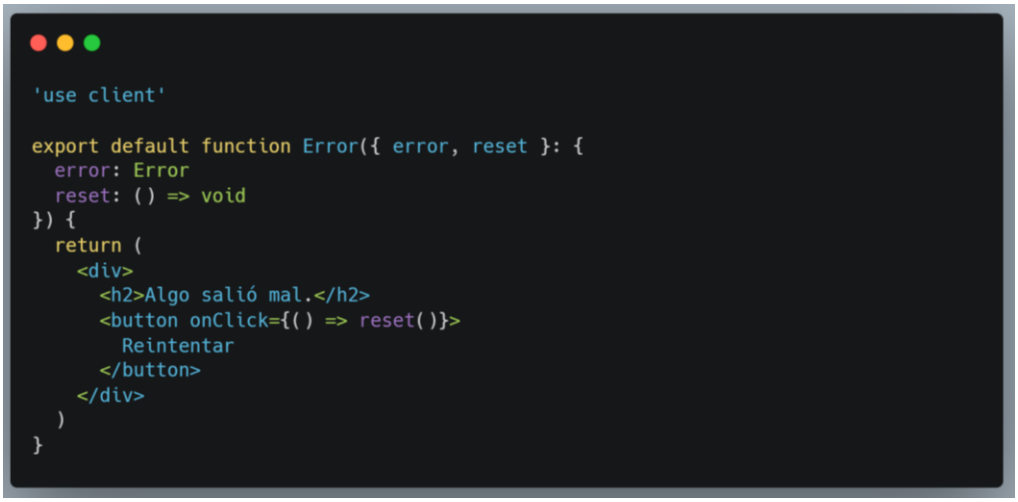
Son fallos del sistema:

- Base de datos caída
- Timeout de red
- Bug en código

Estos deben capturarse y registrarse (logging).

2.2. Implementación de reset() Nextjs

Un error.js puede ser un Client Component:



```
'use client'

export default function Error({ error, reset }: {
  error: Error
  reset: () => void
}) {
  return (
    <div>
      <h2>Algo salió mal.</h2>
      <button onClick={() => reset()}>
        Reintentar
      </button>
    </div>
  )
}
```

Ilustración 7 – reset() vuelve a intentar renderizar el segmento - Fuente: Autor de contenidos

2.3. Validación en Servidor y Anti-Leakage (Zod)

¿Por qué validar en el servidor?

Validar solo en el cliente es como poner una puerta sin llave. El usuario puede modificar el request manualmente usando herramientas como DevTools.

La validación en servidor garantiza:

- Integridad de datos
- Prevención de inyección SQL
- Protección contra datos malformados

El concepto clave aquí es **Sanitización**: limpiar y validar antes de procesar.

Implementación con Zod

Zod es una librería moderna de validación tipada.

```zod
import { z } from 'zod'

export const userSchema = z.object({
  email: z.string().email(),
  password: z.string().min(6)
})
```

Ilustración 8 – Definición del Schema en Zod - Fuente: Autor de contenidos

```server-action
'use server'

import { userSchema } from './schema'

export async function registerUser(formData: FormData) {
  const data = {
    email: formData.get('email'),
    password: formData.get('password')
  }

  const result = userSchema.safeParse(data)

  if (!result.success) {
    return { error: 'Datos inválidos' }
  }

  // Guardar en base de datos
}
```

Ilustración 9 – Validación en una Server Action Zod - Fuente: Autor de contenidos

3. Web Vitals y optimización avanzada (Streaming, Suspense).

Las **Core Web Vitals** son métricas oficiales de Google que miden la experiencia real del usuario. No miden solo velocidad técnica, sino percepción humana: ¿la página carga rápido?, ¿se mueve inesperadamente?, ¿responde al instante?

En la era del **App Router (Next 16)**, muchas optimizaciones vienen por defecto gracias a:

- **React Server Components (RSC)** - menos JavaScript en el cliente.
- **Streaming** - render progresivo.
- **Optimización automática de imágenes y fuentes.**

El objetivo no es solo “cargar rápido”, sino que la aplicación se sienta **instantánea**.

Métrica	Qué mide	Umbral Ideal	Factor de riesgo en Next.js
LCP	Velocidad de carga percibida	< 2.5s	Imágenes sin prioridad o JS pesado
INP	Responsividad a la interacción	< 200ms	Bloqueo del hilo principal por JS

CLS	Estabilidad visual	< 0.1	Fuentes sin swap o imágenes sin dimensiones

Tabla 3 - Métricas de carga en nextjs 16+ - Fuente: Autor de Contenidos

- **LCP (Largest Contentful Paint): Mide cuándo el elemento principal visible (hero image o título grande) termina de cargar. Si el LCP es lento, el usuario percibe que la página está "trabada".**

- **INP (Interaction to Next Paint):** Reemplaza a FID.

Mide cuánto tarda la interfaz en responder a una interacción real (click, input).

- **CLS (Cumulative Layout Shift):** Mide cuánto se mueve el contenido inesperadamente.

3.1. Streaming y Suspense: El fin de los Loading Spinners Globales

En el renderizado tradicional (CSR o SSR clásico), el usuario espera a que TODO termine antes de ver algo. Con Streaming, enviamos HTML en partes (chunks) a medida que están listos. Es como servir comida por tiempos en un restaurante en vez de esperar a tener todos los platos listos.

Estrategia	Experiencia
SSR clásico	Pantalla en blanco hasta terminar
CSR	Descarga JS antes de mostrar contenido
Streaming	Render progresivo inmediato

Tabla 4 - Render Tradicional vs Streaming nextjs 16+ - Fuente: Autor de Contenidos

3.2. Implementación con loading.js

```tsx
// app/dashboard/loading.tsx
export default function Loading() {
  return <div className="skeleton">Cargando dashboard...</div>
}
```

Ilustración 10 – Ejemplo loading.tsx - Fuente: Autor de contenidos

Next.js mostrará este archivo automáticamente mientras el segmento carga.

3.3. Implementación con Suspense Granular



```
import { Suspense } from 'react'
import SlowComponent from './SlowComponent'
import Skeleton from './Skeleton'

export default function Page() {
  return (
    <div>
      <h1>Dashboard</h1>

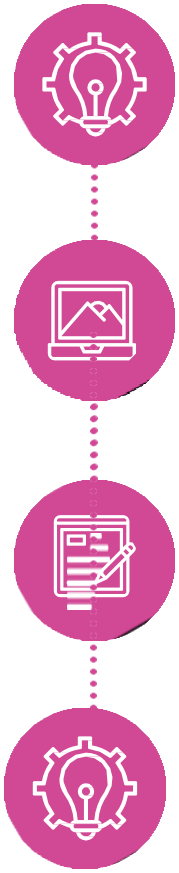
      <Suspense fallback={<Skeleton />}>
        <SlowComponent />
      </Suspense>
    </div>
  )
}
```

Ilustración 11 – Ejemplo suspense granular - Fuente: Autor de contenidos

Bibliografía:

- Google. (2025). *Core Web Vitals*. https://web.dev/vitals/
- Google Chrome Developers. (2026). *Interaction to Next Paint (INP)*. https://web.dev/inp/
- Next.js. (2026). *Next.js documentation (App Router & Server Actions)*. https://nextjs.org/docs
- Next.js. (2026). *Optimizing performance*. https://nextjs.org/docs/app/building-your-application/optimizing
- Next.js. (2026). *Data fetching and caching*. https://nextjs.org/docs/app/building-your-application/data-fetching
- React. (2026). *React Server Components*. https://react.dev/reference/react-server
- React. (2026). *Suspense for data fetching*. https://react.dev/reference/react/Suspense
- OpenTelemetry. (2026). *OpenTelemetry documentation*. https://opentelemetry.io/docs/
- Vercel. (2026). *Streaming and Suspense in Next.js*. https://vercel.com/docs
- Vercel. (2026). *Next.js analytics*. https://vercel.com/docs/analytics
- Zod. (2026). *Zod documentation*. https://zod.dev/
- TypeScript. (2026). *TypeScript handbook*. https://www.typescriptlang.org/docs/
- Google. (2026). *Lighthouse performance auditing*. https://developer.chrome.com/docs/lighthouse/
- Mozilla Developer Network (MDN). (2026). *Web performance fundamentals*. https://developer.mozilla.org/en-US/docs/Web/Performance
- Node.js Foundation. (2026). *Node.js documentation*. https://nodejs.org/docs/latest/api/

Recursos de Apoyo



Video

Video: Master Next.js Server Actions (2026 Guide) - Tutorial práctico de 1h sobre mutaciones, estados pending, revalidación y patrones de formularios en Next.js 16

Material de Lectura

Tutorial Robin Wieruch: Forms con Server Actions - Explica useFormState, useFormStatus, validación y revalidación con ejemplos reales.

Guia

Guía: Forms con Server Actions y Zod - Validación schemas, mensajes consistentes y protección en servidor.

Guia

Guía: Forms con Server Actions y Zod - Validación schemas, mensajes consistentes y protección en servidor.